

ACTIVITY 1

Socratic Prompting for a Selenium Automation Framework

Purpose: Demonstrate how an AI Solution Architect gathers requirements one question at a time before proposing a scalable, data-driven Selenium framework.

Acts as a Solution Architect and gathers requirements using Socratic questioning.

1. User and AI Requirement-Gathering Conversation

Question 1

What type of application will the framework test?

- A. Web application - browser-based system.
- B. Mobile web - website used mainly on phones or tablets.
- C. Desktop application - installed application.
- D. Web and mobile web - both browser experiences.

User

A - Web application.

Question 2

Which browsers must be supported?

- A. Chrome only - simplest maintenance.
- B. Chrome and Edge - common business browsers.
- C. Chrome, Edge and Firefox - wider coverage.
- D. All supported browsers including Safari - widest coverage.

User

C - Chrome, Edge and Firefox.

Question 3

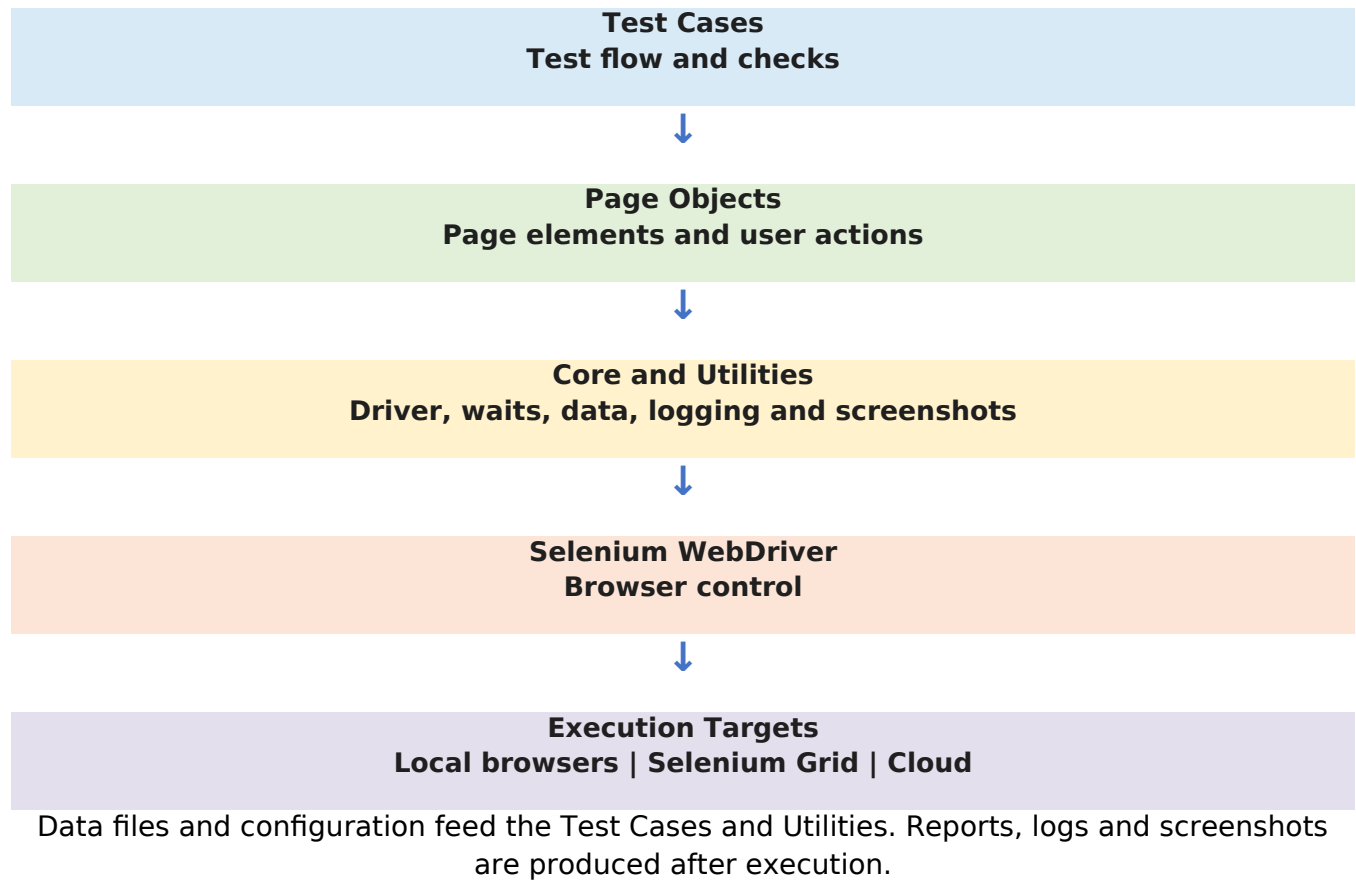
Where should tests run?

- A. Local machines - suitable for development.
- B. Selenium Grid - suitable for parallel execution.
- C. Cloud platform - useful for many browser and OS combinations.
- D. Hybrid - local for development and Grid or cloud for larger runs.

User	D - Hybrid.
Question 4	How should the framework receive test data? A. Excel - easy for business users. B. JSON or CSV - easy to store with source code. C. Database - suitable for centrally managed data. D. Multiple sources - allows Excel, JSON, CSV and database data.
User	D - Multiple sources.
Question 5	Which Java test runner should be used? A. TestNG - supports groups, data providers and parallel runs. B. JUnit - simple and widely used. C. Cucumber - suitable when business-readable scenarios are required. D. No preference - AI may recommend one.
User	A - TestNG.
Question 6	Which design pattern should manage pages and UI actions? A. Page Object Model - one class per page or component. B. Screenplay - models users, tasks and interactions. C. Keyword-driven - test steps use defined keywords. D. No preference - AI may recommend one.
User	A - Page Object Model.
Question 7	What level of parallel execution is required? A. None - tests run one after another. B. Test-level - separate test classes run together. C. Method-level - individual test methods run together. D. Configurable - parallel level and thread count are controlled by configuration.

User	D - Configurable.
Question 8	What reporting is required? A. TestNG report - basic result report. B. Extent Report - readable HTML report with screenshots. C. Allure - dashboard-style reporting and history. D. Extent plus TestNG logs - rich report with standard execution details.
User	D - Extent plus TestNG logs.
Question 9	How should failed tests be handled? A. Screenshot only - capture the failed screen. B. Screenshot and logs - capture evidence and technical details. C. Automatic retry - rerun failed tests. D. Screenshot, logs and limited retry - capture evidence and retry only when configured.
User	D - Screenshot, logs and limited retry.
Question 10	Should the framework run from a CI/CD pipeline? A. No - manual runs only. B. Jenkins - run through Jenkins jobs. C. Azure DevOps - run through Azure Pipelines. D. Tool-neutral - use Maven commands that can run in Jenkins, Azure DevOps or another pipeline.
User	D - Tool-neutral.

Simple Architecture Drawing



2. Answers to the Six Items

1. Summarize my chosen requirements

- The framework will test a web application in Chrome, Edge and Firefox.
- It will run locally during development and on Selenium Grid or a cloud platform for bigger test runs.
- It will use Java, Selenium, TestNG and Maven.
- It will follow Page Object Model and support data from Excel, JSON, CSV and a database.
- It will support configurable parallel execution, reporting, screenshots, logs, retry and CI/CD execution.

2. Explain the architecture and design decisions

- The framework uses separate layers. This keeps the code easy to understand and change.
- Test classes describe what must be tested. Page classes describe how to work with each screen.
- Utility classes handle common work such as browser setup, waits, reading data, screenshots and logs.
- Configuration files control browser, environment, parallel threads and retry settings without changing Java code.
- This separation lowers duplicate code and makes future maintenance easier.

3. Provide the recommended project structure

- src/main/java/pages - Page Object classes
- src/main/java/core - driver and framework setup
- src/main/java/utils - reusable helpers
- src/test/java/tests - TestNG test classes
- src/test/resources/config - environment settings
- src/test/resources/testdata - Excel, JSON and CSV data
- reports - HTML reports, logs and screenshots
- pom.xml - Maven dependencies and build settings
- testng.xml - suites, groups and parallel settings

4. Implement the complete framework step by step with well-commented Java code

- Create a Maven project and add Selenium, TestNG, WebDriverManager, reporting, logging and data-reader dependencies.
- Create a configuration reader for browser, URL, timeout, thread count and execution target.
- Create a DriverFactory that gives each parallel test its own WebDriver using ThreadLocal.
- Create a BaseTest class to start and close the browser safely.
- Create reusable wait, screenshot, logging and test-data utilities.
- Create Page Object classes with locators and simple business actions.
- Create TestNG test classes and DataProviders for data-driven execution.
- Add listeners for reports, screenshots, logs and controlled retry.
- Add testng.xml and Maven commands for local, Grid and pipeline runs.

5. Explain why each design choice was made and discuss alternatives

- Page Object Model was selected because it is simple and keeps page changes away from test logic. Screenplay is an alternative for very large or behavior-focused projects.
- TestNG was selected because it supports groups, DataProviders, listeners and parallel runs. JUnit is a good alternative for simpler projects.

- Maven was selected because it manages libraries and gives one standard command for local and CI/CD execution. Gradle is an alternative build tool.
- Hybrid execution was selected because developers need quick local runs while teams also need scalable Grid or cloud runs.
- Extent Report was selected for easy-to-read HTML output. Allure is an alternative when test history and dashboard-style analysis are important.

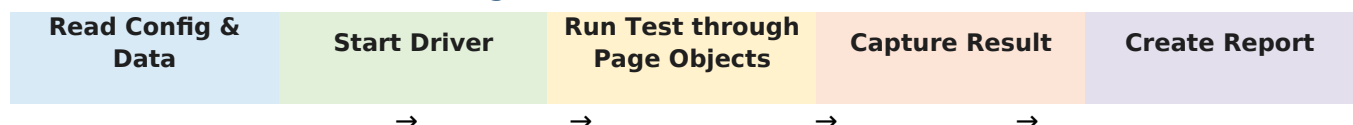
6. Follow industry best practices

- Keep locators inside Page Objects and keep checks inside test classes.
- Do not use fixed sleeps. Use explicit waits for expected conditions.
- Do not hard-code URLs, passwords, browser names or test data.
- Use one WebDriver per parallel thread and always close it after the test.
- Use stable locators such as IDs or dedicated test attributes when available.
- Capture useful failure evidence but avoid retrying every failure automatically.
- Keep tests independent so they can run in any order.
- Run small smoke tests on every build and larger regression suites on a planned schedule.
- Keep dependencies updated and review logs and reports after every execution.

6. Recommended Project Structure Drawing

```
selenium-framework/
|-- pom.xml
|-- testng.xml
|-- src/main/java/
|   |-- core/
|   |-- pages/
|   `-- utils/
|-- src/test/java/tests/
|-- src/test/resources/
|   |-- config/
|   `-- testdata/
`-- reports/
```

7. Execution Flow Drawing



8. Conclusion

The main lesson from this activity is that the AI should not start coding immediately. It should first ask focused questions, one at a time, and allow the user to choose the required options. After the requirements are clear, the AI can propose a framework that is easier to build, maintain and scale.